SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO2 – Manipulation	**Assessment:**	Assessment Tool 1 – ANALYTICAL PROBLEM SOLVING
Weightage:	35% of CIA		
CO2 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	S. Janani Surya	**Reg. No.:**	192325011
Section / Batch:		**Date:**	20/10/26.

Code of Conduct

I S. Janani Surya. (192325[illegible]) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer all five questions. Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
English Word Classes, Part of Speech Tagging	Morphological decomposition of connected, connecting, and connection; identification of roots, suffixes, inflectional vs. derivational forms, and normalization for document indexing.	Q1
English Word Classes, Tagsets for English, Rule-Based Part of Speech Tagging	Morphological parsing of unhappy, happiness, and happily; identification of prefixes, suffixes, base forms, and semantic effects of derivational morphology.	Q2
Counting Words, Part of Speech Tagging, English Word Classes	Stemming and root extraction for played, player, and playing; handling grammatical variations and word formation changes for search preprocessing.	Q3
Finite-State Morphological Analysis, Rule-Based Part of Speech Tagging, Transformation-Based Tagging	Finite-state parsing of writes, writing, and written; modeling regular and irregular verb inflections and root normalization.	Q4
English Word Classes, Rule-Based Part of Speech Tagging, Counting Words	Porter Stemmer-based processing of relational, relation, and relate; application of derivational suffix removal and stem extraction for retrieval.	Q5

Scanned by Camera Scanner

Annexure A – ANALYTICAL PROBLEM SOLVING

1. **Given the input words: "connected", "connecting", "connection", design a morphological analysis pipeline for a document indexing system.**

- Apply rules to decompose each word into root and suffix components.
- Differentiate between inflectional and derivational forms within the dataset.
- Normalize all variants to a common base form for efficient retrieval.
- Determine the parsed structure and normalized output for each word.
- Write a Python program to implement the above morphological analysis pipeline. The program should accept the given input words, perform decomposition, classify each suffix as inflectional or derivational, normalize the words to their common base form, and display the parsed structure and normalized output in a tabular format.

2.Given the input words: "unhappy", "happiness", "happily", design a morphological parsing module for sentiment analysis.

- Identify prefixes, suffixes, and base forms using rule-based decomposition.
- Ensure semantic consistency across derived word forms.
- Classify each transformation as inflectional or derivational.
- Produce the root and morphological breakdown for each word.
- Write a Python program to implement the above morphological parsing module. The program should accept the given input words, identify the prefixes, suffixes, and base forms, classify each transformation as inflectional or derivational, and display the morphological breakdown and normalized root word in a tabular format.

3. Given the input words: "played", "player", "playing", design a stemming-based preprocessing module for a search engine.

- Apply morphological rules to strip affixes and identify the base form.
- Handle both grammatical variations and word formation changes.
- Ensure consistent root extraction across all forms.
- Determine the stem and classification for each input word.
- Develop a Python program for a stemming-based preprocessing module used in a search engine. The program should process the input words "played", "player", "playing" by applying appropriate stemming rules to remove prefixes/suffixes where applicable, identify the stem of each word, distinguish between grammatical inflections and word-formation changes, and generate a structured output showing the original word, extracted stem, removed affix (if any), transformation type (inflectional/derivational), and the final normalized form suitable for indexing and information retrieval.

4. Given the input words: "writes", "writing", "written", design a finite-state morphological parsing module for a text processing system.

- Apply state transitions to represent suffix transformations and irregular forms in verb inflection.
- Differentiate between regular and irregular inflectional patterns within the dataset.
- Ensure consistent mapping of all forms to a common root representation.
- Determine the state transitions, morphological breakdown, and normalized output for each word.
- Develop a Python program that implements a finite-state morphological parser for the input words "writes", "writing", "written". The program should construct a finite-state transition model to process regular suffixes and irregular verb forms, identify the corresponding root word, classify each input as a regular or irregular inflection, trace the

Scanned by Camera Scanner

sequence of state transitions during parsing, and display the state transition path, morphological components, root form, and normalized representation for each input word in a well-formatted output.

5. Given the input words: "relational", "relation", "relate", design a Porter Stemmer-based preprocessing module for document retrieval.

- Apply stemming rules step-by-step to remove derivational suffixes and obtain base forms.
- Handle variations in suffix patterns while preserving semantic consistency.
- Ensure uniform root extraction across all derived forms.
- Determine the stemming steps, intermediate forms, and final stem for each input word.
- Develop a Python program that implements the Porter Stemming algorithm for the input words "relational", "relation", "relate". The program should apply the Porter stemming rules sequentially to remove derivational suffixes, display the intermediate form obtained after each stemming step, handle different suffix patterns while preserving the word meaning, and generate the final stem for each word. The output should clearly present the original word, applied stemming rule(s), intermediate forms, and the final normalized stem in a structured format suitable for document retrieval applications.

Scanned by Camera Scanner

CO-2 AT-1.

1) Morphological Analysis - Document Indexing.

Aim:- To identify roots suffixes and normalized words.

Algorithm:-

Split words, classify suffixes and Normalise to a common roots.

Program:-

```
words = ["connected", connecting, "connection"]
for w in words:
    if w == "connect" r, s, t = "connect", "ed",
        "Inflectional"
    elif w = 'connecting' r, s, t = "connect", "ing"
    'Inflectional'.
    else : r, s, t = "connect", "ion, Derivational".
    Print ( w, r, s, t, 'connect")
```

Output:-

Connected	Connect	ed	Inflectional	Connect.
Connecting	Connect	ing	Inflectional	Connect
Connection	Connect	ion	Derivational	Connect

Scanned by Camera Scanner

2) Morphological Parsing - Sentiment Analysis.

Aim:- To identify prefixes suffixes and root words

Algorithm:-

Separate affixes classify transformations and find the root.

Program:-

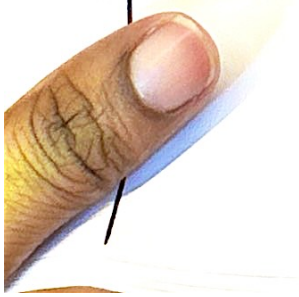
```
words = ["Unhappy", "happiness", "happily"]
for w in words:-
  if w == "unhappy"; p,r,s = "Un", "happy", ""
  elif w == "happiness": p,r,s = "", "happy", "ness"
  else : p, r, s = "", "happy", "ly"
  Print (w,p,r,s, 'derivational', happy"
```

Output :-

Unhappy Un happy derivational happy

happiness happy ness Derivational happy

happily happy ly derivational happy.



Scanned by Camera Scanner

3) Stemming - Search engine:-

Aim:-

To extract common stems from words.

Algorithm:-

Remove suffixes and Normalize words.

Program:-

```
Words = ["played", "player", "playing"]
for w in words:-
    if w == "played": s,t = "play", "Inflectional"
    elif w == "player": s,t = "play", "Derivational"
    else : s.t = "play", "Inflectional"
    Print (w, s,t)
```

Output :-

Played play Inflectional

Player play Derivational

Playing play Inflectional.

Scanned by Camera Scanner

4 Finite-state Morphological Parsing.

Aim:-

To Analyze regular and irregular Verb forms.

Algorithm:-

Map each Verb to its root Using state transitions and normalize it.

Program:-

```
words = {"writes": "write", "writing"}

for w, root in words.item():

  typ = "Irregular" if w == "written" else
  "regular"

  Print (w, root, typ "q0 → q1 → q2").
```

Output:-

writes write Regular q0 → q1 → q2

writing write Regular q0 → q1 → q2.

written write Irregular q0 → q1 → q2.

Scanned by Camera Scanner

5) Porter Stemmer - Document Retrieval.

Aim :-

To remove suffixes and obtain word stems.

Algorithm:-

Apply Porter stemming rules and display the final stems.

Program :-

```
from nltk.stem import PorterStemmer.
Ps = PorterStemmer()
words = ["relational", "relation", "relate"]
for w in words:-
 Print (w, "→", ps.stem(w)).
```

Output :-

```
relational → relat
relation → relat
relate → relat
```

Scanned by Camera Scanner